

PROCESSAMENTO DE LINGUAGEM NATURAL

COM LLMs / 2025PGS2M1

APRESENTAÇÃO

NOSSAS AULAS SERÃO COM ENCONTROS REMOTOS NAS SEGUINTE DATAS:

24/04 E 09/05 NO HORÁRIO DAS 8:00 ÀS 17:15; TEREMOS UM PERÍODO DE ALMOÇO DE UMA HORA E DOIS INTERVALOS DE 15 MINUTOS.

O CONTEÚDO DAS AULAS SERÃO GRAVADOS E O MATERIAL DISPONIBILIZADO PELO CANVAS;

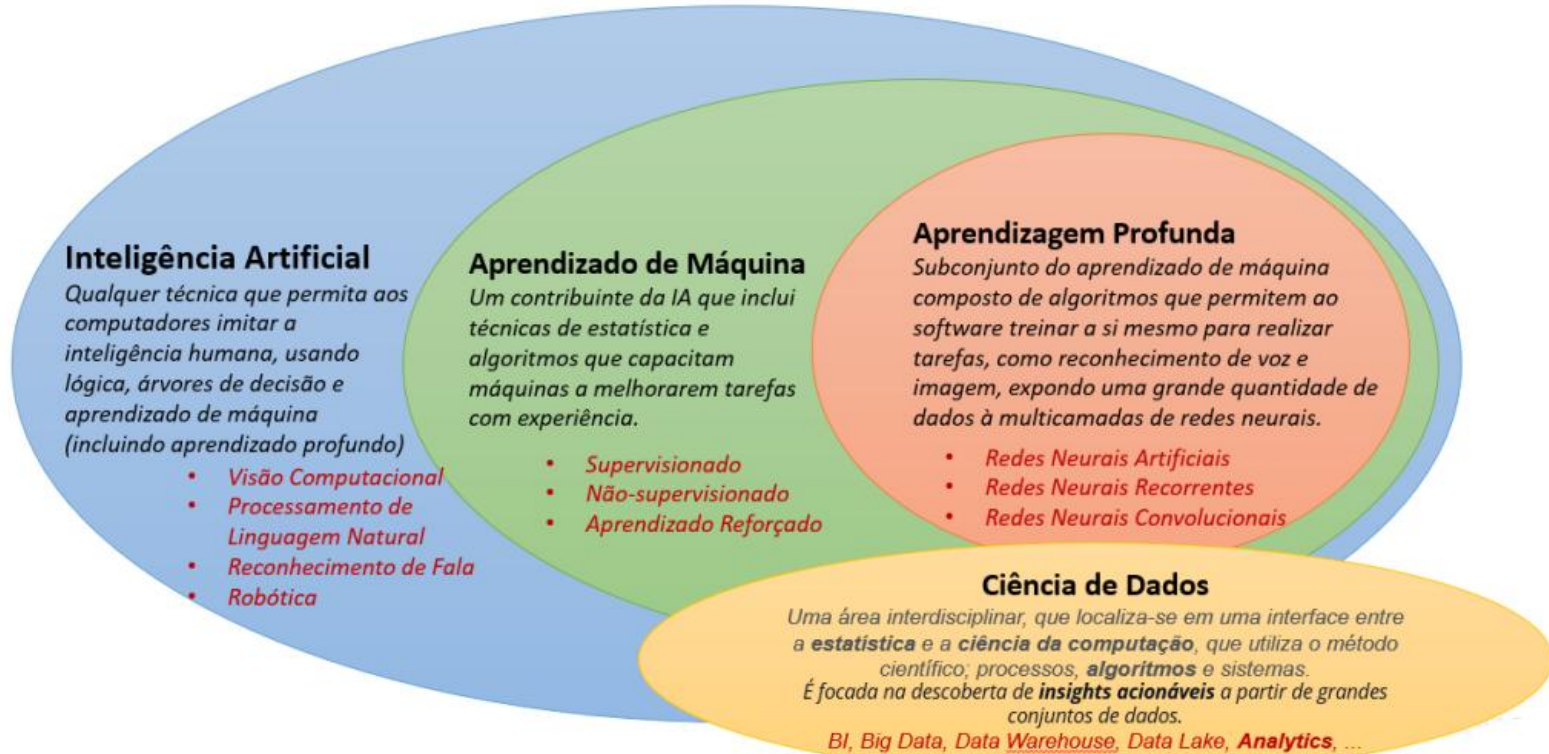
TODOS OS EXERCÍCIOS SERÃO REALIZADOS DURANTE O TEMPO DOS QUATRO ENCONTROS;

TODAS AS ENTREGAS DEVEM SER FEITAS PELO CANVAS;

EXISTE A POSSIBILIDADE DE TERMOS UMA AULA DE MEIO PERÍODO NO DIA 02/05, CONTUDO ESSA AULA NECESSITA CONFIRMAÇÃO DA COORDENAÇÃO;

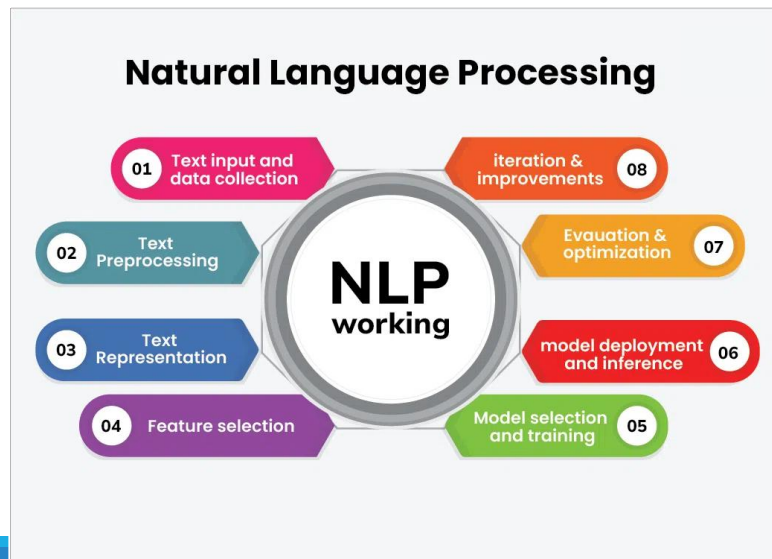
ALÉM DOS EXERCÍCIOS HAVERÁ UMA ENTREGA FINAL.

REVISÃO



REVISÃO

O **Processamento de Linguagem Natural (NLP)** é um campo da **Inteligência Artificial (IA)** que combina **linguística**, **ciência da computação** e **aprendizado de máquina** para permitir que máquinas compreendam, interpretem e gerem linguagem humana (texto ou voz). Seu objetivo é criar sistemas capazes de se comunicar de forma natural com pessoas, automatizando tarefas que dependem da linguagem.



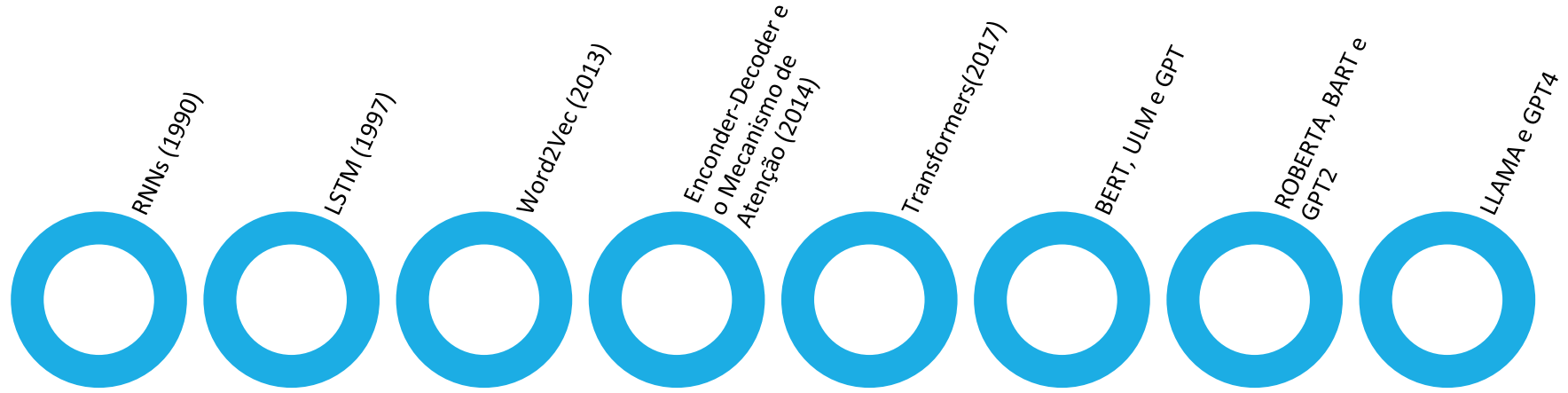
REVISÃO

Redes Neurais Convolucionais (CNNs): São redes neurais especializadas para processamento de imagens. Elas têm uma arquitetura que explora a estrutura espacial dos dados, fazendo uso de camadas convolucionais, de pooling e totalmente conectadas para extrair características relevantes das imagens.

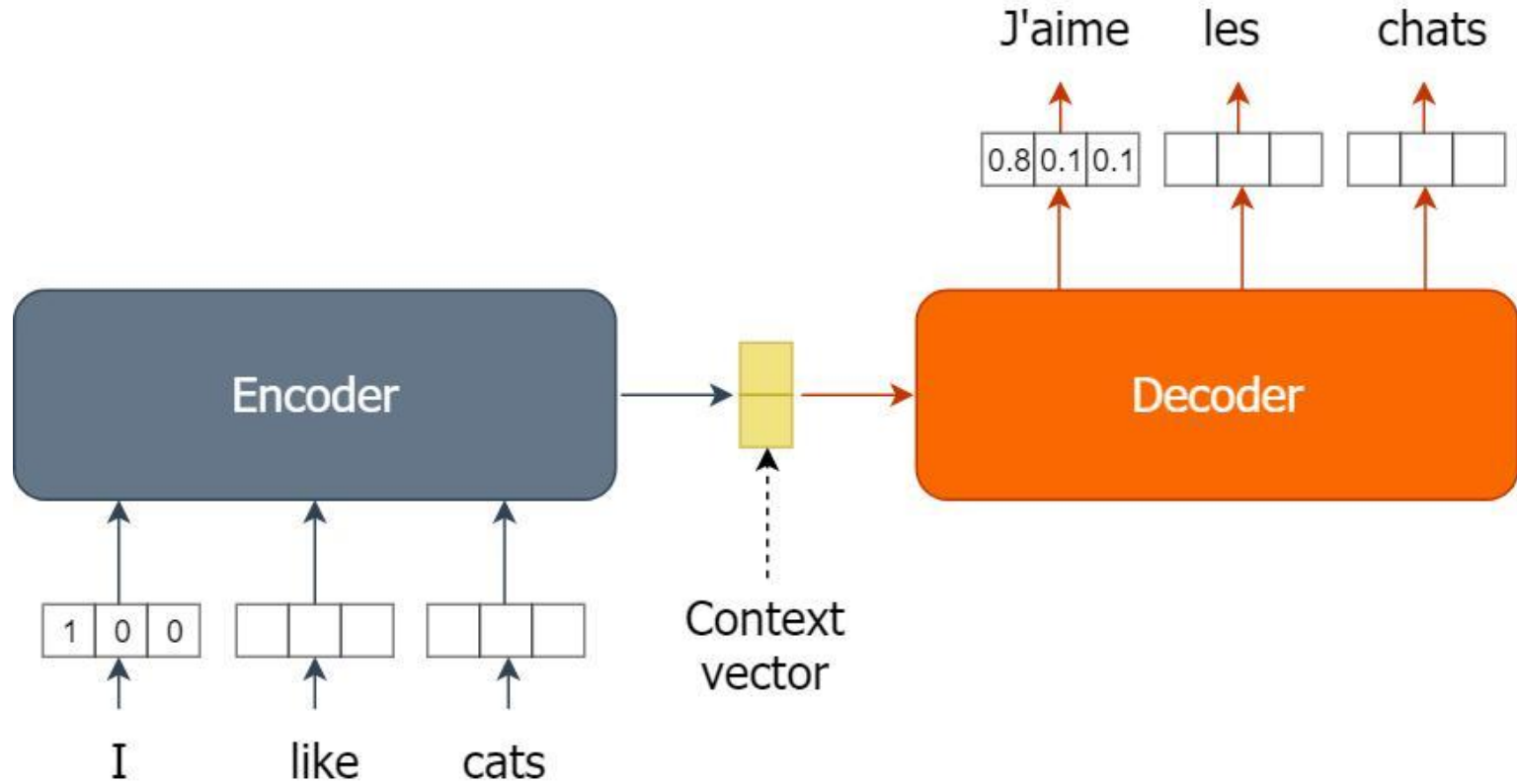
A relação entre Redes Neurais Recorrentes (RNNs) e Large Language Models (LLMs) é de evolução tecnológica e superação de limitações. As RNNs, junto com suas variantes como LSTMs, foram as precursoras no processamento de linguagem natural, enquanto os LLMs modernos utilizam a arquitetura Transformer, que superou as limitações das RNNs para lidar com textos longos e treinamento em larga escala.

O modelo Transformers são um tipo avançado de arquitetura de rede neural, introduzida em 2017, que revolucionou o Processamento de Linguagem Natural (PLN) e IA Generativa. Eles funcionam processando dados sequenciais (como textos) paralelamente, usando um mecanismo de "autoatenção" para entender o contexto e a relação entre palavras distantes, sendo a base de modelos como GPT, BERT e Gemini.

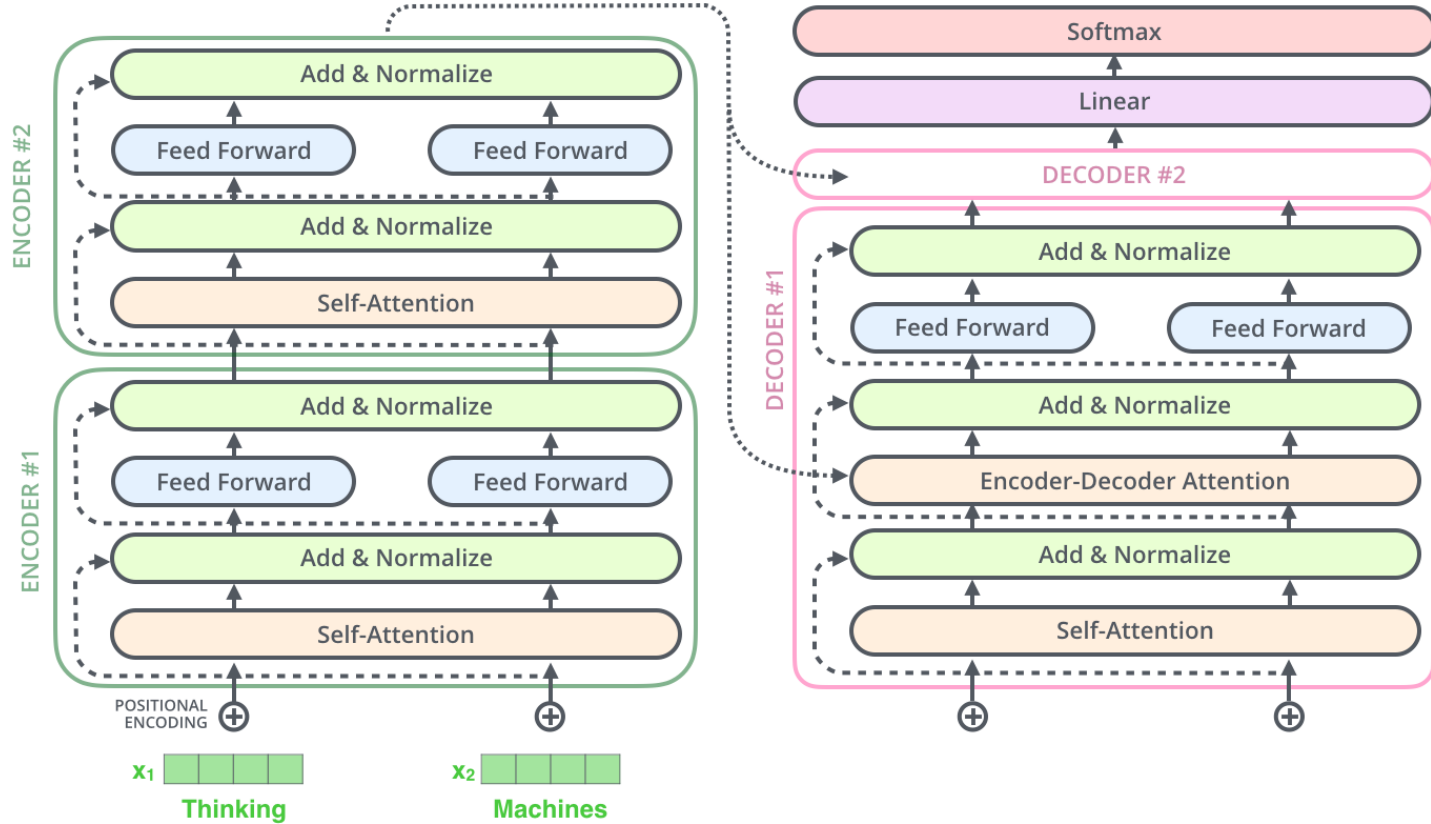
REVISÃO



REVISÃO



REVISÃO



REVISÃO

TÉCNICA	DESCRIÇÃO	APLICAÇÃO EM CHATBOTS
TOKENIZAÇÃO	DIVIDIR FRASES EM PALAVRAS OU SENTENÇAS	SEPARAR COMANDOS DO USUÁRIO PARA ANÁLISE
STEMMING	REDUZIR PALAVRAS À SUA RAIZ (EXEMPLO: “CORRENDO” – “CORR”)	MELHORAR A EFICIÊNCIA DO MODELO DE CHATBOT
LEMATIZAÇÃO	SIMILAR AO STEMMING, MAS MANTENDO SIGNIFICADO GRAMATICAL CORRETO	INTERPRETAR MELHOR A INTENÇÃO DO USUÁRIO
RECONHECIMENTO DE ENTIDADES (NER)	IDENTIFICAR NOMES, DATAS, LOCAIS EM UM TEXTO	EXTRAIR INFORMAÇÕES DE MENSAGENS DO USUÁRIO
POS TAGGING (PART-OF-SPEECH TAGGING)	IDENTIFICA A FUNÇÃO GRAMATICAL DAS PALAVRAS	MELHOR INTERPRETAÇÃO DE PERGUNTAS COMPLEXAS EM UM TRANSAÇÕES
TF-IDF (TERM FREQUENCY – INVERSE DOCUMENT FREQUENCY)	MEDE A IMPORTANCIA DE PALAVRAS DENTRO DE UM CONJUNTO DE TEXTOS	MELHORAR RESPOSTAS DE CHATBOTS A PERGUNTAS SOBRE DOCUMENTOS LONGOS

REVISÃO

Análise Morfológica

- **Part-of-Speech Tagging (POS):** Identifica a classe gramatical de cada palavra (substantivo, verbo, adjetivo, etc.). *Exemplo:* Na frase "**O gato preto correu rápido**", as tags seriam: "**gato**" → substantivo, "**preto**" → adjetivo, "**correu**" → verbo.

Exemplo Prático

Frase: "A Amazon anunciou hoje um novo serviço em Londres, que custará US\$ 10 por mês."

Análise Morfológica (Part-Of-Speech):

"Amazon" → substantivo próprio, "anunciou" → verbo, "hoje" → advérbio, "Londres" → substantivo próprio.

REVISÃO

Análise Semântica

- **Named Entity Recognition (NER):** Identifica entidades nomeadas, como pessoas, organizações, datas e locais. *Exemplo:* Em "**João trabalha na Google em São Paulo desde 2020**", reconhece: "**João**" (pessoa), "**Google**" (organização), "**São Paulo**" (local), "**2020**" (data).
- **Resolução de Referência:** Determina a qual entidade pronomes ou expressões se referem. *Exemplo:* Em "**Ana comprou um livro. Ela adorou.**", "**Ela**" refere-se a "**Ana**".

Exemplo Prático

Frase: "A Amazon anunciou hoje um novo serviço em Londres, que custará US\$ 10 por mês."

Análise Semântica (Named Entity Recognition):

Entidades: "Amazon" (organização), "Londres" (local), "US\$ 10" (valor monetário).

REVISÃO

A visão geral evolutiva das aplicações que utilizam inteligência artificial, o início foi proporcionado pelas NLP Clássicas na qual utiliza regras e processos estatísticos. Com a chegada das Deep Learning os problemas de classificação e seq2seq foram solucionados. Com a revolução com o uso dos Transformers dificuldades anteriores foram superadas e os modelos tinham condições de geração de contexto.

Atualmente o uso das LLMs foca em criação de agentes multimodais o que direcionam para uma geração de sistemas cognitivos híbridos.

Entre os exemplos que tivemos observamos o funcionamento de códigos que realizaram classificação de sentimentos e modelos de tradutores. Apesar da complexidade encontrada nesses dois tipos de aplicações os modelos de agentes virtuais de atendimento (chatbots) são os que carregam maior nível de dificuldade em seu desenvolvimento. Hoje conheceremos sobre os desafios e complexidades na criação e aplicação desses modelos.

REVISÃO

Para o desenvolvimento de um tradutor a complexidade fica em compreender contexto, preservar significado mantendo sua gramática e adaptando a expressões culturais. Trabalhando com múltiplos idiomas e preservando a intenção.

Exemplo: É quando o usuário digita “Break a leg” e a tradução se torne “Quebre uma perna” e não “Boa sorte”.

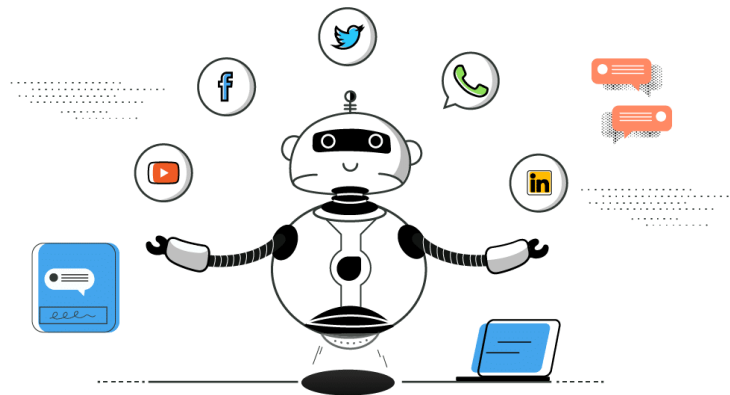
Aqui temos a necessidade de modelagem semântica, para que tenha relações contextuais com alinhamento linguístico e geração textual coerente. Sendo que o grau de dificuldade aumenta porque, idiomas possuem estruturas diferentes, onde pode ter problemas culturais. Assim, existindo dependência contextual e com o uso de textos longos geram perda semântica. Apesar da complexidade de cada um dos exemplos a situação de um chatbot e grau elevado se comparado os dois anteriores. Isso é o que veremos a seguir:

MODELAGEM DE AGENTES INTELIGENTES

Os chatbots são sistemas de software projetados para interagir com humanos em linguagem natural, seja por texto ou voz. Eles são amplamente utilizados em diversas áreas, como atendimento ao cliente, suporte técnico, e-commerce e até mesmo em assistentes pessoais virtuais. A evolução dos chatbots tem sido impulsionada pelo avanço das técnicas de Machine Learning (ML) e Processamento de Linguagem Natural (NLP), permitindo que esses sistemas se tornem mais inteligentes e capazes de entender e responder de forma mais natural e contextualizada.

MODELAGEM DE AGENTES INTELIGENTES

Um chatbot trabalha com a interpretação de linguagem, com a geração de textual e a classificação de intenções. Esse modelo contém uma memória contextual que tem como obrigação o gerenciamento de fluxos, entre suas etapas de processamento temos a recuperação de informações, raciocínio, tomada de decisão, personalização e manutenção de estado da conversa. Desta forma temos dois principais tipos de modelos de chatbots, que são:



REVISÃO

Nos exemplos de aplicação, conhecemos o funcionamento de sistemas de classificação de emoções, seu funcionamento é focado em receber um texto e retornar em uma das categorias a seguir: Alegria; Tristeza; Raiva; Medo; Neutro.

Um exemplo é quando o usuário digita: “Hoje foi um dia horrível.”. Na avaliação devemos receber algo como tristeza/raiva.

A complexidade é menor porque existe um conjunto limitado de classes, desta forma temos um problema que é supervisionado. Isso normalmente não exige memória conversacional e pode funcionar apenas com inferência simples. Desta forma, seus modelos são menores e o contexto é curto.

O maior desafio para este modelo costuma ser a ambiguidade emocional, a ironia, sarcasmo e as emoções mistas.

MODELOS DE CHATBOTS

Chatbots Baseados em Regras

Esses chatbots seguem um conjunto predefinido de regras e scripts. Eles são programados para reconhecer palavras-chave específicas e responder com respostas pré-determinadas. As vantagens em utilizar esse modelo fica por conta de serem simples de implementar e eficazes para tarefas específicas e previsíveis. Contudo, **são limitados** em termos de flexibilidade e capacidade de lidar com conversas complexas ou fora do escopo das regras definidas. Não aprendem com as interações.



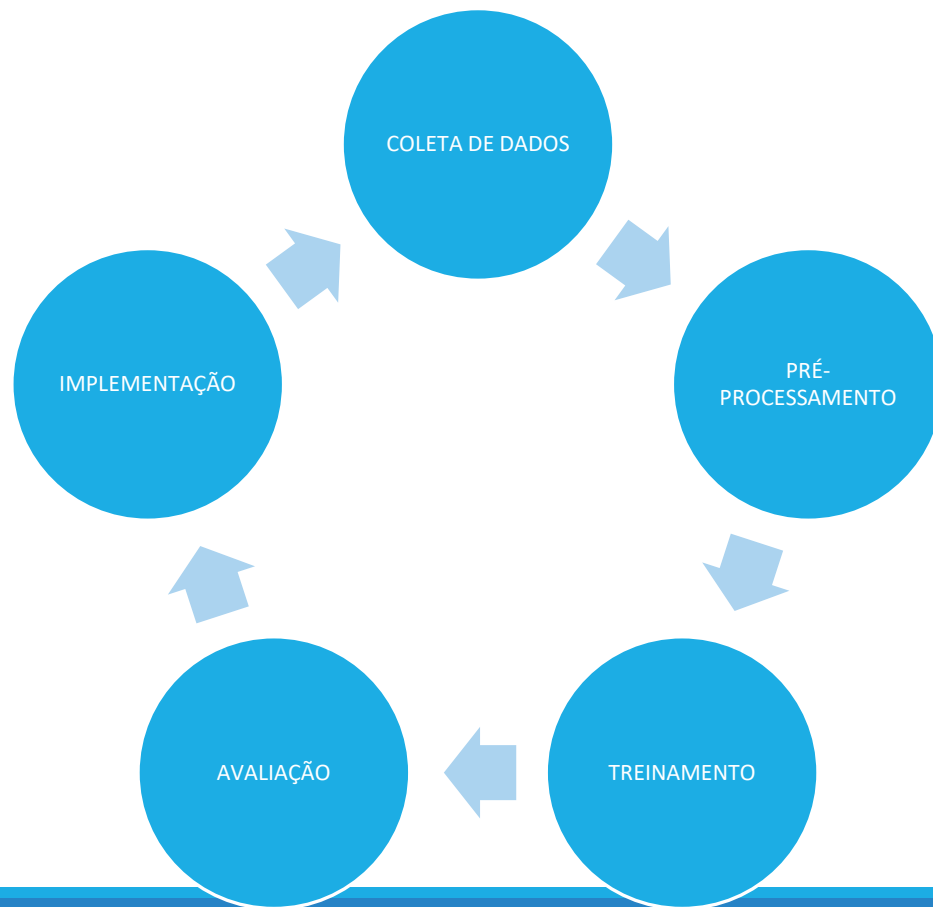
MODELOS DE CHATBOTS

Chatbots Baseados em Machine Learning

Esses chatbots utilizam algoritmos de ML para aprender a partir de dados. Eles podem entender o contexto, a intenção do usuário e gerar respostas mais naturais e adaptativas. São mais flexíveis e capazes de lidar com uma variedade de cenários e linguagens naturais. Melhoram com o tempo à medida que são expostos a mais dados. Entretanto, requerem grandes volumes de dados para treinamento e podem ser mais complexos de implementar e manter.



CICLO DE VIDA CHATBOT ML



CICLO DE VIDA CHATBOT ML

Coleta de Dados:

- **Objetivo:** Reunir um conjunto de dados representativo das interações que o chatbot terá.
- **Fontes:** Logs de conversas anteriores, FAQs, transcrições de atendimento ao cliente, etc.
- **Desafios:** Garantir a qualidade, diversidade e volume suficiente de dados.

Pré-processamento:

- **Limpeza:** Remover ruídos, como erros de digitação, símbolos desnecessários, etc.
- **Tokenização:** Dividir o texto em palavras ou frases menores.
- **Normalização:** Converter texto para minúsculas, remover stopwords, lematização, etc.
- **Vectorização:** Converter texto em representações numéricas (e.g., TF-IDF, Word Embeddings).

CICLO DE VIDA CHATBOT ML

Treinamento:

- **Seleção do Modelo:** Escolher algoritmos de ML adequados (e.g., redes neurais, SVM, etc.).
- **Treinamento:** Alimentar o modelo com dados pré-processados para que ele aprenda a mapear entradas (perguntas) para saídas (respostas).
- **Ajustes:** Fine-tuning de hiperparâmetros e otimização do modelo.

Avaliação:

- **Métricas:** Acurácia, precisão, recall, F1-score, BLEU (para geração de texto), etc.
- **Testes:** Realizar testes com dados não vistos durante o treinamento para avaliar a generalização do modelo.
- **Feedback:** Coletar feedback de usuários reais para identificar áreas de melhoria.

CICLO DE VIDA CHATBOT ML

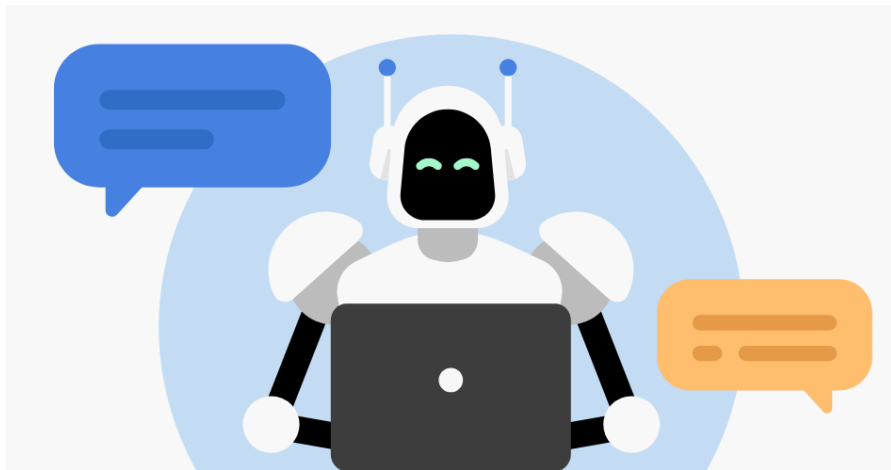
Implementação:

- **Integração:** Implementar o chatbot em plataformas de mensagens, websites, ou aplicativos.
- **Monitoramento:** Acompanhar o desempenho em tempo real e coletar dados de novas interações.
- **Manutenção:** Atualizar o modelo com novos dados e ajustes conforme necessário para manter a eficácia.



CLASSIFICAÇÕES DE INTENÇÕES

A classificação de intenções é uma tarefa fundamental no desenvolvimento de chatbots. Ela envolve a identificação da intenção por trás de uma mensagem do usuário, o que permite ao chatbot responder de forma adequada. Por exemplo, em uma mensagem como "Qual é o saldo da minha conta?", a intenção pode ser classificada como "consultar_saldo".



CLASSIFICAÇÕES DE INTENÇÕES

Conceito de intenção

Uma intenção (intent) é: a representação da ação, necessidade ou objetivo do usuário dentro da conversa.

Exemplo simples

Usuário escreve: "Como rastrear meu pedido?"

O chatbot não está interessado apenas nas palavras.

Ele tenta descobrir: "O que o usuário quer?"

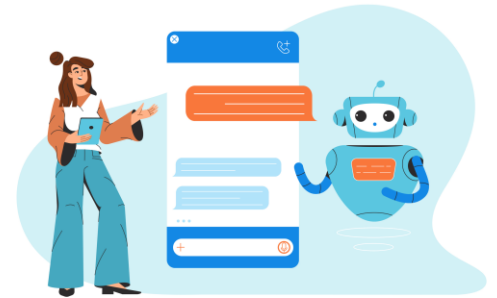
Neste caso: acompanhar entrega; localizar pedido.

Então a intenção identificada é: rastreamento

CLASSIFICAÇÕES DE INTENÇÕES

Como os Chatbots Identificam a Intenção do Usuário

1. **Entrada do Usuário:** O chatbot recebe uma mensagem de texto ou voz.
2. **Pré-processamento:** A mensagem é limpa e preparada para análise (tokenização, normalização, etc.).
3. **Classificação de Intenção:** Um modelo de Machine Learning é usado para prever a intenção do usuário com base no texto processado.
4. **Resposta:** Com base na intenção identificada, o chatbot gera uma resposta apropriada ou executa uma ação específica.



CLASSIFICAÇÕES DE INTENÇÕES

Técnicas para Treinamento de Modelos de Classificação de Intenções

Uma das técnicas é o uso combinado do **Bag of Words (BoW)** que representa o texto como um vetor de contagens de palavras, ignorando a ordem das palavras e do **TF-IDF (Term Frequency-Inverse Document Frequency)** que melhora o **BoW** ponderando as palavras pela sua frequência no documento e inversamente pela sua frequência no corpus. Sua vantagem é que é simples e eficaz para textos curtos. As limitações ficam por conta de ignorar a semântica e a ordem das palavras.

Outra técnica é **Word Embeddings** que também é a combinação do **Word2Vec** e **GloVe**. O primeiro gera vetores densos que capturam relações semânticas entre palavras. Já o **GloVe** é similar ao Word2Vec, mas baseado em co-ocorrência global de palavras. Suas vantagens são a captura semântica e as relações entre palavras. As limitações ficam por conta de depender de grandes volumes de dados para treinamento.

CLASSIFICAÇÕES DE INTENÇÕES

Técnicas para Treinamento de Modelos de Classificação de Intenções

É possível utilizar modelos clássicos de ML como o **Naïve Bayes** que é baseado no teorema de Bayes, assume independência entre as features. O modelo de **Random Forest** que utiliza um conjunto de árvores de decisão que melhora a generalização. O **SVM (Support Vector Machines)** que encontra o hiperplano que melhor separa as classes.

A vantagem da aplicação desses modelos é que todos são simples, consolidados e eficazes para uso de base de dados (datasets) menores. Suas limitações ficam por não poderem capturar a complexidade de textos longos.

```
1 import numpy as np
2 from sklearn.feature_extraction.text import TfidfVectorizer
3 from sklearn.naive_bayes import MultinomialNB
4 from sklearn.pipeline import make_pipeline
5
6 # Dados de treinamento
7 frases = [
8     "Qual é o horário de funcionamento?",
9     "Vocês estão abertos aos domingos?",
10    "Quais são os preços dos produtos?",
11    "Como faço para cancelar um pedido?",
12    "Onde está minha entrega?",
13    "Quais são os métodos de pagamento?",
14    "Vocês aceitam cartão de crédito?",
15    "Como posso falar com o suporte?",
16    "Quais são as promoções atuais?",
17    "Como posso rastrear meu pedido?"
18 ]
19
20 intencoes = [
21     "horario",
22     "horario",
23     "preco",
24     "cancelamento",
25     "rastreamento",
26     "pagamento",
27     "pagamento",
28     "suporte",
29     "promocoes",
30     "rastreamento"
31 ]
32
33 # Criando o modelo TF-IDF + Naïve Bayes em um pipeline
34 vectorizer = TfidfVectorizer()
35 X = vectorizer.fit_transform(frases)
36 modelo = MultinomialNB()
37 modelo.fit(X, intencoes)
38
```

```

39 # Dicionário de respostas para cada intenção
40 respostas = {
41     "horario": "Nosso horário de funcionamento é das 8h às 18h, de segunda a sexta.",
42     "preco": "Os preços variam conforme o produto. Você pode conferir no nosso site.",
43     "cancelamento": "Para cancelar um pedido, acesse 'Meus Pedidos' na sua conta.",
44     "rastreamento": "Para rastrear seu pedido, utilize a seção 'Rastrear Pedido' em nosso site.",
45     "pagamento": "Aceitamos cartão de crédito, débito e boleto bancário.",
46     "suporte": "Você pode entrar em contato com nosso suporte pelo telefone 0800-123-456.",
47     "promocoes": "Atualmente, temos promoções especiais disponíveis no site."
48 }
49
50 # Função para prever a intenção corretamente
51 def prever_intencao(pergunta):
52     pergunta_tfidf = vectorizer.transform([pergunta]) # Transformando a nova frase em vetor TF-IDF
53     intencao = modelo.predict(pergunta_tfidf)[0] # Fazendo a predição
54     return respostas.get(intencao, "Desculpe, não entendi sua pergunta.")
55
56 # Loop de interação com o usuário
57 print("Olá! Sou o assistente virtual. Como posso ajudar você hoje? (Digite 'sair' para encerrar)")
58 while True:
59     pergunta = input("Você: ")
60     if pergunta.lower() == 'sair':
61         print("Assistente: Obrigado por conversar comigo. Até logo!")
62         break
63     resposta = prever_intencao(pergunta)
64     print(f"Assistente: {resposta}")
65

```

CLASSIFICAÇÕES DE INTENÇÕES

Neste exemplo temos uma arquitetura clássica de NLP e Machine Learning tradicional, baseada em:

- TF-IDF que utilizamos para vetorização textual;
- Naive Bayes utilizamos como classificador probabilístico;
- Intent Classification aplicado para identificador de intenções;
- Chatbot baseado em regras onde temos respostas fixas.

Seu uso é prático e atende necessidades básicas com desafios de respostas e conversação limitada.

CLASSIFICAÇÕES DE INTENÇÕES

Técnicas para Treinamento de Modelos de Classificação de Intenções

Com o avanço das tecnologias e técnicas o uso de modelos modernos foram aplicados para chatbots entre essas técnicas são:

- **BERT (Bidirectional Encoder Representations from Transformers):** Modelo de linguagem pré-treinado que captura contexto bidirecional.
- **GPT (Generative Pre-trained Transformer):** Modelo de linguagem que gera texto de forma autônoma.
- **Transformers:** Arquitetura que usa mecanismos de atenção para capturar dependências de longo alcance.

O uso dessas técnicas são da alta precisão e capacidade de capturar contexto complexo. Contudo, requer grande poder computacional e grandes volumes de dados. Assim veremos o próximo conteúdo:

```

1 from transformers import pipeline
2 import numpy as np
3 from sklearn.metrics.pairwise import cosine_similarity
4 from sentence_transformers import SentenceTransformer
5
6 # ♦ Criando o modelo de classificação de intenções usando BERT
7 modelo_transformer = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
8
9 # ♦ Modelo de embeddings para fallback (caso a predição do transformer falhe)
10 modelo_embeddings = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
11
12 # ♦ Intenções conhecidas e suas respostas
13 intencoes = {
14     "horario": "Nosso horário de funcionamento é das 8h às 18h, de segunda a sexta.",
15     "preco": "Os preços variam conforme o produto. Você pode conferir no nosso site.",
16     "cancelamento": "Para cancelar um pedido, acesse 'Meus Pedidos' na sua conta.",
17     "rastreamento": "Para rastrear seu pedido, utilize a seção 'Rastrear Pedido' em nosso site.",
18     "pagamento": "Aceitamos cartão de crédito, débito e boleto bancário.",
19     "suporte": "Você pode entrar em contato com nosso suporte pelo telefone 0800-123-456.",
20     "promocoes": "Atualmente, temos promoções especiais disponíveis no site."
21 }
22
23 # ♦ Lista de intenções para o modelo Transformers reconhecer
24 lista_intencoes = list(intencoes.keys())
25
26 # ♦ Gerando embeddings das intenções para o fallback
27 intencoes_embeddings = modelo_embeddings.encode(lista_intencoes)
28
29 # ♦ Função para prever a intenção corretamente
30 def prever_intencao(pergunta):
31     # ♦ Primeiro tentamos classificar com o modelo transformer
32     resultado = modelo_transformer(pergunta, lista_intencoes)
33
34     # ♦ Obtém a intenção com maior pontuação
35     intencao_predita = resultado['labels'][0]
36     score = resultado['scores'][0]
37

```



```

37
38 # ♦ Se a confiança for alta (> 0.5), retorna a resposta correspondente
39 if score > 0.5:
40     return intencoes.get(intencao_predita, "Desculpe, não entendi sua pergunta. Poderia reformular?")
41
42 # ♦ Se a confiança for baixa, usamos um fallback baseado em Similaridade de Cosseno
43 pergunta_embedding = modelo_embeddings.encode([pergunta])
44 similaridades = cosine_similarity(pergunta_embedding, intencoes_embeddings).flatten()
45 indice_mais_proximo = np.argmax(similaridades)
46
47 # ♦ Se a similaridade for razoável (> 0.4), usa essa intenção
48 if similaridades[indice_mais_proximo] > 0.4:
49     intencao_predita = lista_intencoes[indice_mais_proximo]
50     return intencoes[intencao_predita]
51
52 # ♦ Se nenhuma estratégia funcionar, retorna a resposta padrão
53 return "Desculpe, não entendi sua pergunta. Poderia reformular?"
54
55 # ♦ Loop de interação com o usuário
56 print("Olá! Sou o assistente virtual. Como posso ajudar você hoje? (Digite 'sair' para encerrar)")
57 while True:
58     entrada = input("Você: ")
59
60     if entrada.lower() == "sair":
61         print("Chatbot: Foi um prazer conversar com você. Até logo!")
62         break
63
64     resposta = prever_intencao(entrada)
65     print(f"Chatbot: {resposta}")
66

```

CLASSIFICAÇÕES DE INTENÇÕES

O exemplo apresentado implementa um sistema híbrido de classificação de intenções para chatbot utilizando modelos Transformers, embeddings semânticos e similaridade vetorial. Trata-se de uma arquitetura relativamente moderna para NLP, combinando dois paradigmas distintos de IA:

- classificação zero-shot com LLM/Transformer;
- recuperação semântica por embeddings.

Aqui temos o uso de cinco componentes que são a classificação zero-short que identifica a intenção usando transformer, o sentence embeddings que é a representação vetorial semântica, a similaridade de cosseno que mede a proximidade semântica, o sistema de fallback que executa uma recuperação alternativa e loop conversacional no qual simula o chatbot.

CLASSIFICAÇÕES DE INTENÇÕES

A principal diferença entre os dois exemplos está no paradigma de Inteligência Artificial utilizado para compreender a linguagem. Embora ambos implementem um chatbot baseado em intenções, eles pertencem a gerações diferentes do NLP (Natural Language Processing). Podemos resumir assim:

Exemplo	Tecnologia principal	Geração
TF-IDF + Naive Bayes	NLP estatístico clássico	NLP tradicional
Transformers + Embeddings	NLP neural moderno	IA generativa / LLM

Dentro desses exemplos, as intenções representam o objetivo ou a finalidade da mensagem enviada pelo usuário. Em um chatbot baseado em NLP, a intenção é a categoria semântica que descreve o que o usuário deseja fazer ou perguntar.

ATIVIDADE

Ao término deste conteúdo o aluno deverá realizar a leitura da atividade individual com o título: IMPLEMENTAÇÃO DE CHATBOTS UTILIZANDO NLP CLÁSSICO E TRANSFORMERS. Para esta atividade será necessário o uso de outros recursos disponíveis dentro do conteúdo da aula.

DÚVIDAS ou PERGUNTAS?



MUITO OBRIGADO!!!!



daniel.ohata@facens.br

REFERÊNCIAS

BISONG, Ekaba; BISONG, Ekaba. Google colabatory. **Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners**, p. 59-64, 2019.

DE ANDRADE, Amanda Figueiredo et al. A ÉTICA NO USO DA INTELIGÊNCIA ARTIFICIAL E SEUS RISCOS JURÍDICOS. *Revista Acadêmica Online*, v. 11, n. 56, p. e1400-e1400, 2025.

ISZCZUK, Ana Claudia Duarte et al. Evoluções das tecnologias da indústria 4.0: dificuldades e oportunidades para as micro e pequenas empresas. **Brazilian Journal of Development**, v. 7, n. 5, p. 50614-50637, 2021.

MARCONI, Francesco. Artificial Intelligence Backbone. < <https://twitter.com/fpmarconi/status/794208040207740928> >, 2016 Acesso 01/01/2019.

MICHALSKI, Ryszard Stanislaw; CARBONELL, Jaime Guillermo; MITCHELL, Tom M. (Ed.). **Machine learning: An artificial intelligence approach**. Springer Science & Business Media, 2013.

ROSÁRIO, João Maurício. **Automação industrial**. Editora Baraúna, 2012.

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. Malaysia; Pearson Education Limited, 2016.

SANTOS, Gustavo Soares. Novas Tecnologias Aplicadas na Construção Civil: Conceitos da Indústria 4.0. *RCT-Revista de Ciência e Tecnologia*, v. 8, 2022.

ZHOU, Zhi-Hua. **Machine learning**. Springer nature, 2021.